

6장 합성곱 신경망 II (2)

🕒 생성일 @2025년 4월 27일 오후 5:22

1 이미지 분류를 위한 신경망

5. ResNet

2 객체 인식을 위한 신경망

1. R-CNN

2. 공간 피라미드 풀링

3. Fast R-CNN

4. Faster R-CNN

3 이미지 분할을 위한 신경망

1. 완전 합성곱 네트워크

2. 합성곱 & 역합성곱 네트워크

3. U-Net

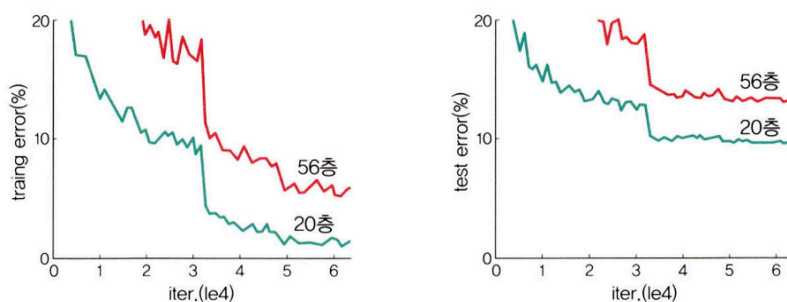
4. PSPNet

5. DeepLabv3/DeepLabv3+

1 이미지 분류를 위한 신경망

5. ResNet

- ResNet 핵심은 깊어진 신경망을 효과적으로 학습하기 위한 방법으로 레지듀얼(residual) 개념을 고안한 것임
- 신경망은 깊이가 깊어질 수록 성능이 좋아지다가 일정 단계에 다다르면 오히려 성능이 나빠짐

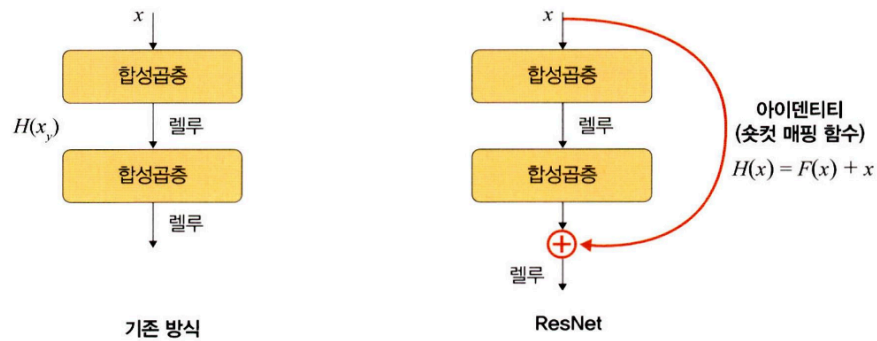


네트워크 56층이 20층보다 더 나쁜 성능을 보임

→ ResNet은 이러한 문제를 해결하기 위해 레지듀얼 블록을 도입, 레지듀얼 블록은 기울기가 잘 전파될 수 있도록 일종의 숏컷을 만들어 줌

⇒ 숏컷을 두어 기울기 소멸문제를 방지

- ResNet 구조



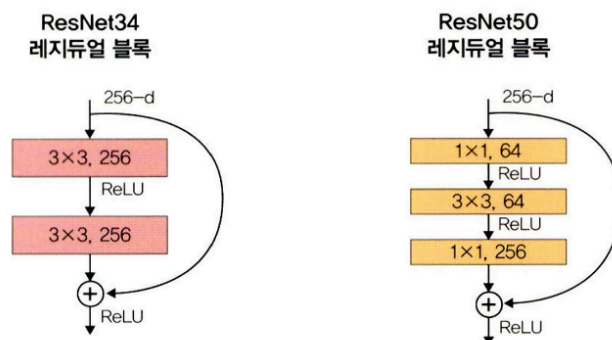
- 블록 : 계층의 묶음, 합성곱층을 하나로 묶은 것
- 레지듀얼 블록



- 레지듀얼 블록 여러개 쌓은 것 → ResNet
- but, 계층 수 ↑ ⇒ 파라미터 수 증가 문제

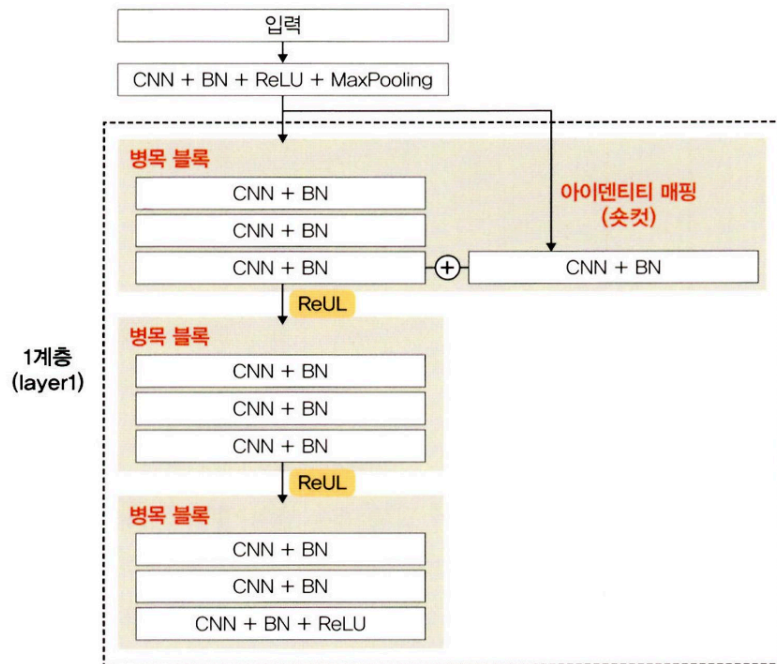
: 계층이 깊어질수록 파라미터 증가 → 계층의 깊이가 깊어질수록 파라미터는 무제한으로 커질 것

⇒ 해당 문제 해결 위해 병목 블록 둬, 계층이 깊어져도 파라미터 수는 감소 ex. ResNet50



- 병목블록 : 파라미터수가 감소하는 효과
→ 1 X 1 합성곱층의 채널 수를 조절해 차원을 줄였다 늘리는 것 가능

- 아이덴티티 매핑 (숏컷, 스킵연결): 입력 x 가 어떤 함수를 통과하더라도 다시 x 라는 형태로 출력되도록 함



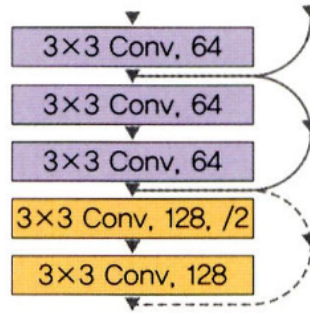
```
def forward (self, x):
    1=X
    x= self.conv1(x) x= self.bn1(x)
    x= self.relu(x) x= self.conv2(x) x= self.bn2(x)
    if self.downsample is not None:
        i =self.downsample(i) # 다운샘플적용

    x =+ I # 아이덴티티매핑적용
    x = self.relu(x)
    return x
```

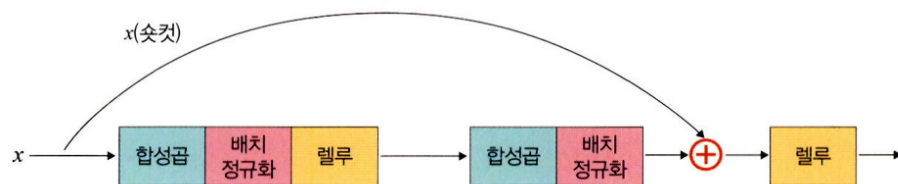
- 입력 x 를 1라는 변수에 저장 → 입력 x 는 합성곱층을 통과하다가 마지막 x 에 I 를 더해줌

ex. x 가(28, 28, 64)라고 가정, x 를 i 변수에 저장했기 때문에(28, 28, 64)가 될 것임 → 합성곱층을 통과 하면서 같은 형태를 더하기 때문에 최종형태는(28, 28, 64) 그대로가 될 것임

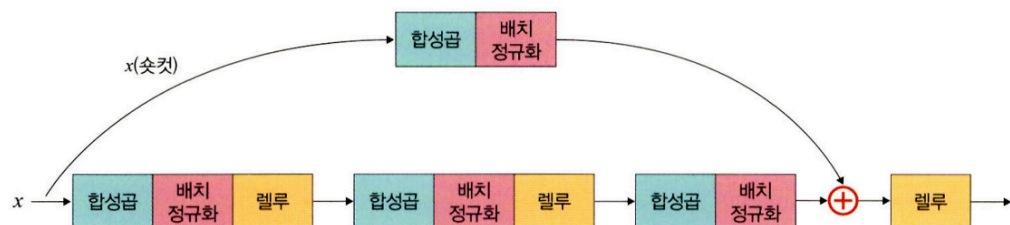
- 다운 샘플: 특성맵 크기를 줄임 (= 풀링)



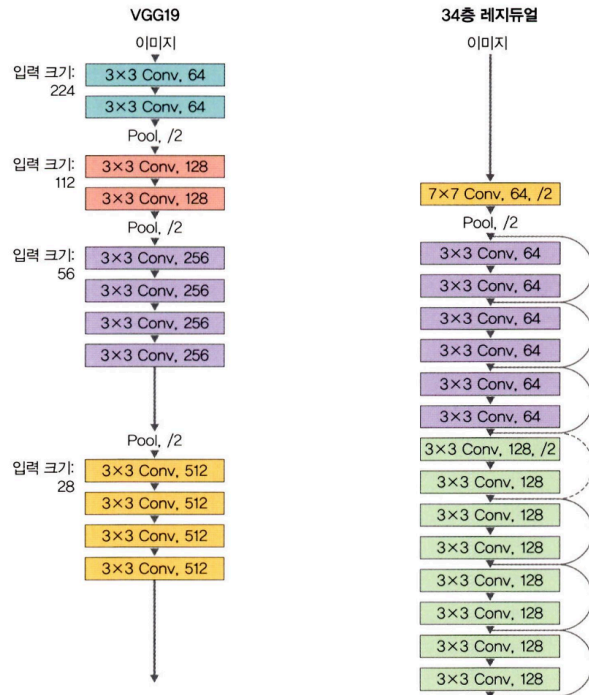
- 보라색 영역의 첫번째 블록에서 특성맵의 형상이(28, 28, 64)였다면 세번째 블록의 마지막 합성곱층을 통과하고 아이덴티티 매핑 까지 완료된 특성맵의 형상도 (28, 28, 64)
- 노란색 영역의 시작 지점에서는 채널수가 128로 늘어남 + /2 : 첫번째 블록에서 합성곱층의 스트라이드가 2로 늘어나(14, 14, 128)로 바뀜
 - ⇒ 보라색과 노란색의 형태가 다름 → 이들 간의 형태를 맞추지 않으면 아이덴티티 매핑을 할 수 없게됨 ⇒ 아이덴티티에 대해 다운샘플 필요
- 입력과 출력의 형태를 똑같이 맞추기 위해서는 스트라이드 2를 가진 1X1 합성곱 계층을 하나 연결 해주면 됨
 - 아이덴티티 블록: 입력과 출력의 차원이 같은 것



- 합성곱 블록 (= 프로젝션 숏컷) : 입력과 출력의 차원이 동일하지 않고 입력의 차원을 출력에 맞추어 변경해야 하는 것



⇒ ResNet은 기본적으로 VGG19 구조 + 합성곱층들을 추가해서 깊게 만듦 + 숏컷들을 추가 = 사실상전부



코드 6-65 필요한 라이브러리 호출

1 네임드튜플(`namedtuple`): 파이썬 자료형 中 하나, 튜플의 성질 + 인덱스 뿐만 아니라 키 값으로 데이터 접근 가능

```
from collections import namedtuple
Student = namedtuple('Student', ['name', 'age', 'DOB']) ----- 네임드튜플 정의
S = Student('홍길동', '19', '187') ----- 네임드튜플에 값 추가

print("The Student age using index is : ", end="")
print(S[1]) ----- 인덱스를 이용한 데이터 접근

print("The Student name using keyname is : ", end="")
print(S.name) ----- 키 값을 이용한 데이터 접근
```

```
The Student age using index is : 19
The Student name using keyname is : 홍길동
```

코드 6-66 이미지데이터 전처리

코드 6-67 변수에 대한 값 정의

코드 6-68 훈련과 테스트 데이터셋 불러오기

코드 6-69 데이터셋을 훈련,검증,테스트 용도로 분리

코드 6-70 이미지에 대한 레이블구분

코드 6-71 이미지 데이터셋 정의

코드 6-72 데이터셋의 데이터를 메모리로 불러오기

코드 6-73 기본 블록 정의

- 1 다운샘플(downsample)이 적용되는 부분, 입력 데이터 크기와 네트워크 통과 후 출력 데이터 크기가 다를 경우 사용함

```
conv = nn.Conv2d(in_channels, out_channels, kernel_size=1, stride=1, bias=True)
```

- 2 아이덴티티 매핑이 적용되는 부분, 특정 층에 존재하는 출력 결과를 그 다음 합성곱층을 통과한 출력결과에 더해주므로 스킵연결이라고도 함

```
def forward(self, x):
    i=x
    x= self.conv1(x)
    x = self.bn1(x)
    x = self.relu(x)
    x= self.conv2(x)
    x= self.bn2(x)
    x = self.relu(x)
    x = self.conv3(x)
    x= self.bn3(x)

    if self.downsample is not None:
        i = self.downsample(i)

    x += i
    x= self.relu(x)
    return x
```

x에 conv1, bn1, relu, conv2, bn2, relu, conv3, bn3값이 더해지다가 초기의 x가 다시 더해 지는 것 ⇒ 아이덴티티매핑 / 숏컷 / 스킵연결

코드 6-74 병목 블록 정의

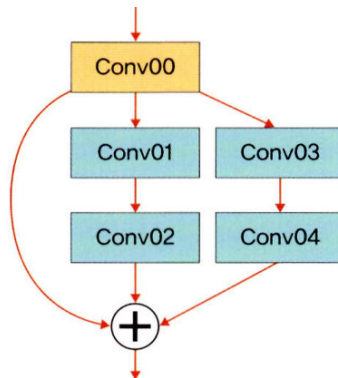
- 기본블록 ↔ 병목블록 차이

- 기본블록: 3x3 합성곱층 2개
 - 병목블록: 1x1 합성곱층, 3x3 합성곱층, 1x1 합성곱층 구조
 - 기본블록 → 병목블록 교체 이유
 - 계층을 깊게 쌓으면서 계산 비용 ↓
 - 계층 ↑ = 활성화함수 기존보다 ↑ = 처리가능한 비선형성 ↑ = 다양한 입력 데이터에 대한 처리 가능
- ⇒ 아이덴티티 매핑과 병목 블록으로 ResNet 네트워크에 더욱 깊은 계층 쌓을 수 있게 됨

코드 6-75 ResNet 모델 네트워크

1

- 각 레지듀얼 분기(residual branch)에 있는 마지막 BN(Batch Normalization)을 0으로 초기화해서 다음 레지듀얼 분기를 0에서 시작할 수 있도록 함
- BN을 0으로 초기화할 경우, 모델 성능이 0.2~0.3%정도 향상
- 레지듀얼 분기: 프로그램 조건에 따라 A,B,C 등으로 분기하는 것



코드 6-76 ResNetConfig 정의

코드 6-77 기본 블록을 사용하여 ResNetConfig 정의

코드 6-78 병목 블록을 사용하여 ResNetConfig 정의

코드 6-79 사전훈련된 ResNet 모델 사용

코드 6-80 사전훈련된 ResNet 모델 확인

코드 6-81 ResNet50 Config를 사용한 ResNet 모델 사용

코드 6-82 옵티마이저와 손실함수 정의

코드 6-83 모델학습 정확도 측정함수 정의

1

`tensor.topk` : `torch.argmax`와 같은 효과, 주어진 텐서에서 가장 큰 값의 인덱스를 얻기 위해 사용, 네트워크 출력에서 가장 확률이 높은 값의 인덱스 반환

```
import torch
x = torch.arange(1., 6.)
print(x)
print('-----')
print(torch.topk(x, 3)) ----- x 입력에서 가장 큰 값 세 개를 선택하여 그 값과 인덱스를 출력
```

```
tensor([1., 2., 3., 4., 5.])
-----
torch.return_types.topk(
  values = tensor([5., 4., 3.]),
  indices = tensor([4, 3, 2]))
```

2

t() : 차원 0과 1을 전치하겠다는 의미

```
x = torch.randn(3) ----- 1차원 텐서 생성
print(x)
print(torch.t(x)) ----- 1차원 그대로 값을 반환
print('-----')
x = torch.randn(2, 3) ----- (2, 3)차원 텐서 생성
```

```
print(x)
print(torch.t(x)) ----- 입력이 전치되어 (3, 2)차원 반환
```

```
tensor(0.7357)
tensor(0.7357)
-----
tensor([[ -0.3894,  0.6999, -0.6733],
        [ 1.7850,  0.1961,  0.2701]])
tensor([[ -0.3894,  1.7850],
        [ 0.6999,  0.1961],
        [-0.6733,  0.2701]])
```

3

텐서 비교 함수

- torch.eq : 텐서가 서로 같은지를 비교

- torch.eq(비교 대상 텐서, 비교 할 텐서)

```
torch.eq(torch.tensor([[1, 2], (3, 4)]), torch.tensor([[1, 1], [4, 4]]))
```

```
# 출력 결과
```

```
# tensor ([[ True, False], [False, True]])
```

- torch.ne : 텐서가 서로 다른지를 비교
- torch.ge : 텐서가 서로 크거나 같은지를 비교

코드 6-84 모델 학습 함수 정의

코드 6-85 모델 평가 함수 정의

코드 6-86 모델 학습 시간 측정 함수 정의

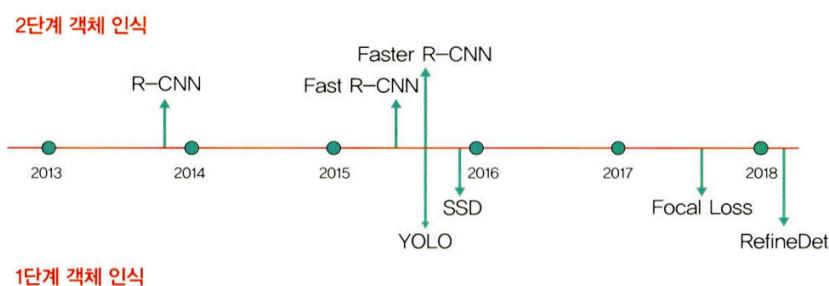
코드 6-87 모델 학습

코드 6-88 테스트 데이터셋을 이용한 모델 예측

코드 6-89 모델 예측에 대한 결과 출력

2 객체 인식을 위한 신경망

- 객체 인식
 - 이미지나 영상 내에 있는 객체를 식별하는 컴퓨터 비전 기술
 - 이미지·영상 내에 있는 여러 객체에 대해 각 개체가 무엇인지 분류하는 문제와 그 객체 위치가 어디인지 박스로 나타내는 위치 검출 (localization) 문제를 다루는 분야
- ⇒ 객체 인식 = 여러 가지 객체에 대한 분류 + 객체의 위치 정보를 파악하는 위치 검출
- 객체 인식 알고리즘 = 1단계 객체인식 / 2단계 객체인식

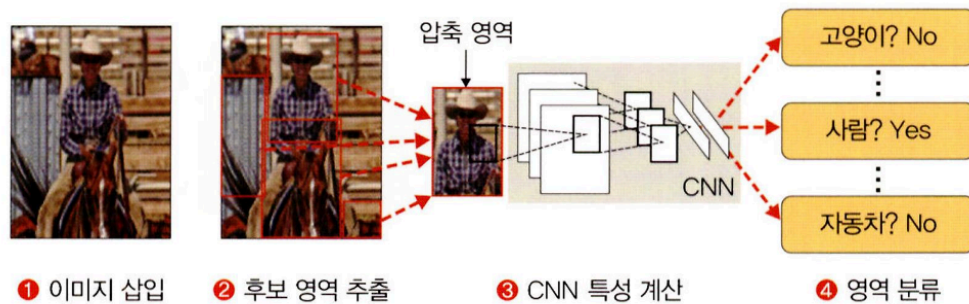


- 1단계 객체 인식: 분류+위치 검출 두 문제 **동시에** 행하는 방법
 - 비교적 빠르지만 정확도 낮음
 - YOLO (You Only Look Once) 계열과 SSD 계열 등이 포함
- 2단계 객체 인식: 분류+위치 검출 두 문제 **순차적으로** 행하는 방법
 - 비교적 느리지만 정확도 높음
 - CNN을 처음으로 적용시킨 R-CNN계열이 대표적

1. R-CNN

- 예전: 슬라이딩 윈도우(sliding window)방식 채택 - 일정한 크기를 가지는 윈도우를 가지고 이미지의 모든 영역을 탐색하면서 객체를 검출해내는 방식
- 알고리즘의 비효율성 때문에 사용 ↓

- 현재 : 선택적 탐색 (selective search) / 알고리즘을 적용한 후보영역(region proposal) 사용
- R-CNN : **CNN** (이미지 분류를 수행) + **후보 영역 알고리즘** (이미지에서 객체가 있을 만한 영역을 제안)
- R-CNN 학습 절차



1. 이미지를 입력으로 받음
2. 2000개의 바운딩 박스를 **선택적 탐색** 알고리즘으로 추출 후 잘라내고, CNN 모델에 넣기 위해 같은크기 (227*227 픽셀)로 통일
3. 크기가 동일한 이미지 2000개에 각각 CNN 모델 적용
4. 각각 분류 진행 후 결과 도출



선택적 탐색

: 객체 인식이나 검출을 위한 가능한 후보영역(객체가있을만한위치, 영역)을 알아내는 방법, 분할 방식을 이용하여 시드(seed)를 선정하고, 그 시드에 대한 완전 탐색을 적용

- 선택적 탐색 과정

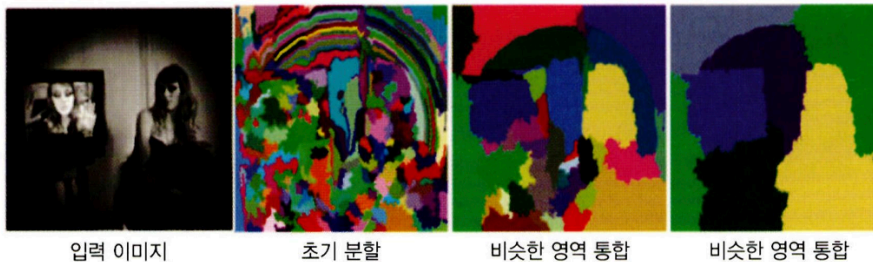
1. 초기 영역 생성 (sub-segmentation)

각각의 객체가 영역 한 개에 할당될 수 있도록 많은 초기영역을 생성함.
즉, 입력된이미지를 영역 다수 개로 분할하는 과정



2. 작은 영역의 통합

1단계에서 영역 여러개로 나눈 것들을 비슷한 영역으로 통합하는데, 이때 탐욕(gr eedy)알고리즘'을 사용하여 비슷한 영역이 하나로 통합 될 때까지 반복함



3. 후보 영역 생성

2단계에서 통합된 이미지들을 기반으로 다음 그림과 같이 후보영역(바운딩박스)을추출



- 관련 용어

- 완전 탐색(exhaustive search): 후보가 될 만한 대상의 크기 및 비율이 모두 다른 상황을 고려하여 후보영역을 찾는 기법
- 분할(segmentation): 영상데이터의 특성(색상, 모양, 무늬등)에 따라 분할하여 후보영역을 선정하는 기법
- 후보 영역 (바운딩 박스): 3D 객체의 형태를 모두 포함할 수 있는 최소 크기의 박스
- 시드(seed): 영상에서는 특정 기준점의 픽셀에서 점점 의미가 같은 영상 범위 까지 픽셀을 확장해 나가면서 분할하는데, 이때 특정 기준점이 되는 픽셀

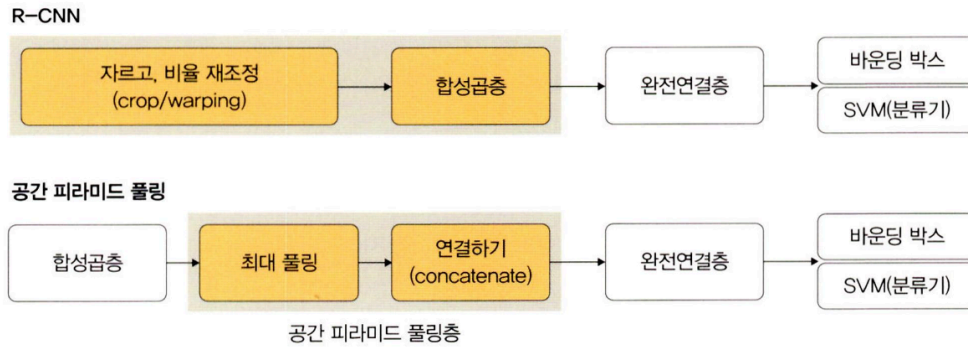
- R-CNN이 크게 발전하지 못한 단점

1. 세단계의 복잡한 학습과정
2. 긴 학습시간과 대용량 저장공간
3. 객체검출(object detection) 속도 문제

→ 이를 해결하기 위해 Faster R-CNN이 생김

2. 공간 피라미드 풀링

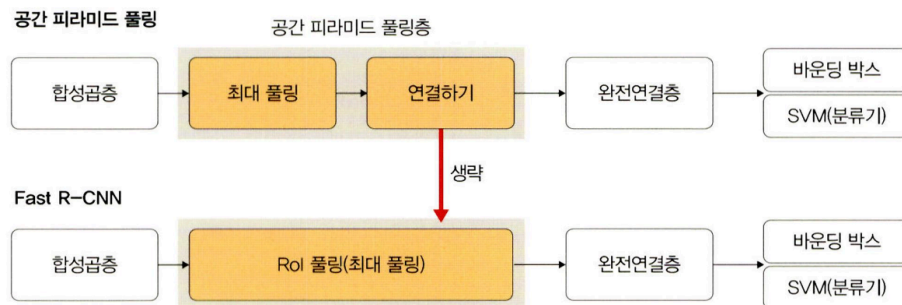
- 기존 CNN 구조들은 모두 완전연결층을 위해 입력이미지를 고정해야 했음
 - 신경망을 통과시키려면 이미지를 고정된 크기로 자르거나 비율 조정(warp) 필요
 - 물체의 일부분이 잘리거나 본래의 생김새와 달라지는 문제 발생
 - ⇒ 이를 해결하고자 공간 피라미드 풀링 도입



- 공간 피라미드 풀링: 입력 이미지 크기에 관계없이 합성곱층 통과, 완전연결층 전달되기 전에 특성맵들을 동일한 크기로 조절해주는 풀링층 적용하는 기법
 - 입력 이미지의 크기를 조절하지 않고 합성곱층을 통과시키기 때문에 원본 이미지의 특징이 훼손 되지 않는 특성맵을 얻을 수 있음
 - 이미지 분류나 객체 인식 같은 여러작업에 적용할 수 있다는 장점

3. Fast R-CNN

- R-CNN : 바운딩 박스 마다 CNN을 돌리고, 분류를 위한 긴 학습시간이 문제
 - Fast R-CNN: R-CNN의 속도문제를 개선하기 위해 RoI풀링 도입
 - ⇒ 선택적 탐색에서 찾은 바운딩박스 정보가 CNN을 통과하면서 유지되도록 하고 최종 CNN 특성맵은 풀링을 적용하여 완전연결층을 통과하도록 크기를 조정



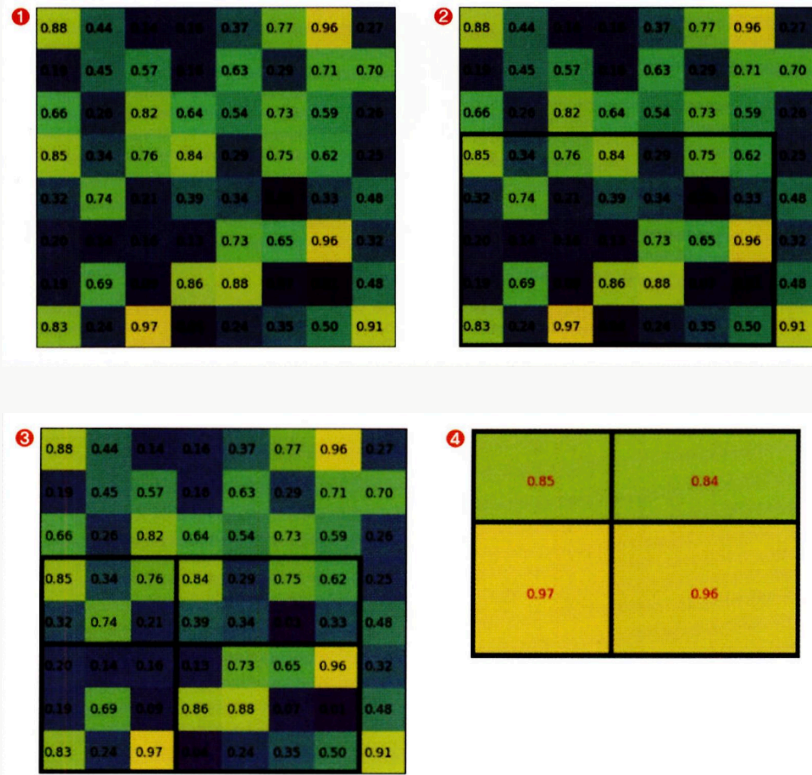


RoI 풀링

: 크기가 다른 특성 맵의 영역마다 스트라이드를 다르게 최대 풀링을 적용하여 결과값 크기를 동일하게 맞추는 방법

if. 박스 한 개 = 픽셀 한 개를 뜻하는 특성맵

1. 8X8특성맵
2. (1)에서 선택적 탐색으로 뽑아냈던 7X5 후보영역
3. (2)를 2x2로 만들기 위해 스트라이드($7/2=3, 5/2=2$)로 풀링영역 정함
4. (3)에 최대풀링을 적용하면 2X2 결과를 얻음

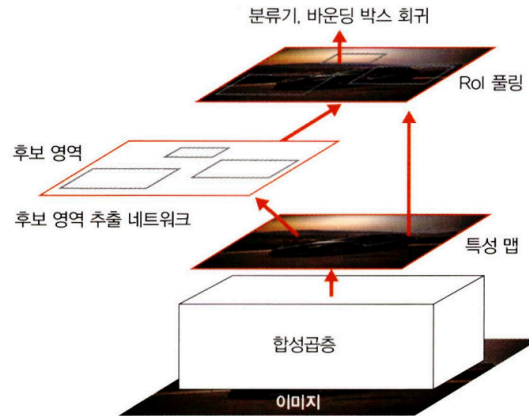


4. Faster R-CNN

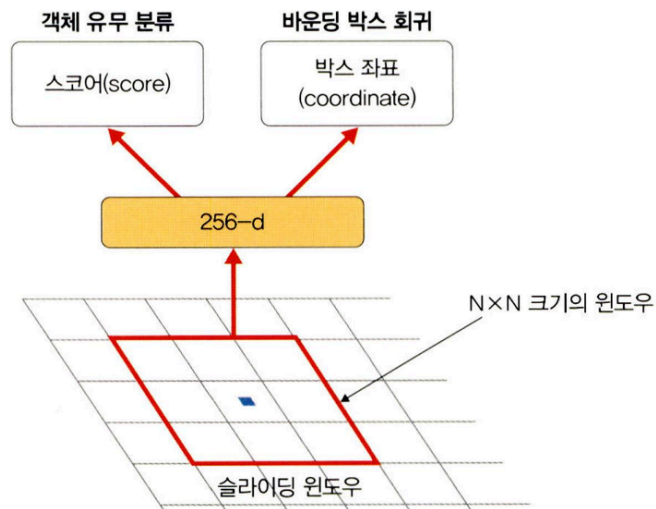
: '더욱 빠른' 객체 인식을 수행하기 위한 네트워크. 기존 Fast R-CNN 속도의 걸림돌이었던 후보 영역 생성을 CNN 내부 네트워크에서 진행할 수 있도록 설계

= FasterR-CNN은 기존 Fast R-CNN에 후보 영역 추출 네트워크를 추가한 것이 핵심, FasterR-CNN에서는 외부의 느린 선택적 탐색 (CPU 계산) 대신 내부의 빠른 RPN (GPU 계산)을 사용

- RPN 구조

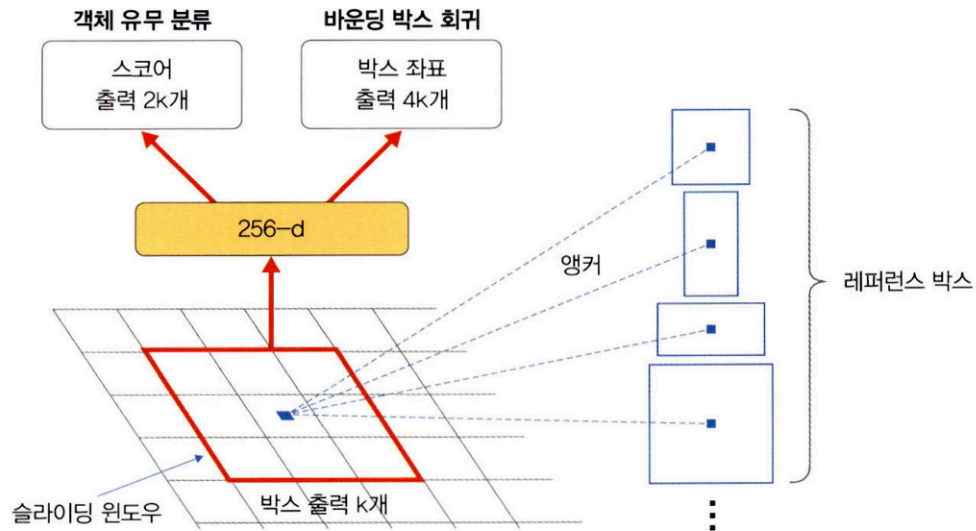


- 합성곱층 마지막에 RPN 위치 → RoI 풀링 + 분류기 → 바운딩 박스 회귀
- 후보 영역 추출 네트워크
 - 특성맵 NxN 크기의 작은 윈도우영역을 입력으로 받고, 해당영역에 객체 존재 유무 판단 위해 이진분류를 수행하는 작은 네트워크 생성
 - 바운딩 박스 회귀: 위치 보정을 위해 추가
 - 슬라이딩 윈도우 방식 → 작은 윈도우 영역(NxN 크기) 이용하여 객체 탐색
: 하나의 특성맵에서 모든 영역에 대한 개체 존재 유무 확인하기 위함



- but, 후보 영역 추출 네트워크 단점 : 이미지에 존재하는 객체들의 크기와 비율이 다양하기 때문에 고정된 NxN 크기의 입력 만으로 다양한 크기와 비율의 이미지를 수용하기 어려움
→ 앵커 : 여러 크기와 비율의 레퍼런스 박스 k개를 미리 정의하고 각각의 슬라이딩 윈도우 위치마다 박스 k개를 출력하도록 설계하는 방식

⇒ 후보 영역 추출 네트워크의 출력값은 모든 앵커 위치에 대해 각각 객체와 배경을 판단하는 2k개의 분류에 대한 출력 + x,y,w,h 위치 보정 값을 위한 4k개의 회귀 출력 가짐

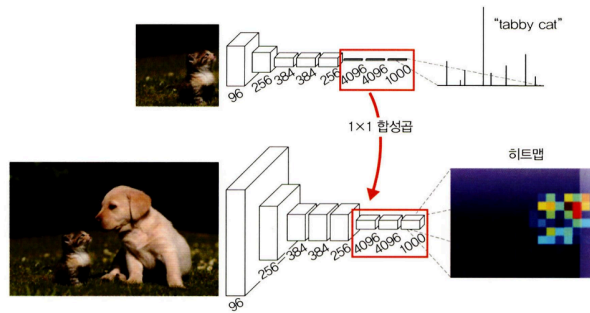


3 이미지 분할을 위한 신경망

- 이미지 분할 : 신경망을 훈련시켜 이미지를 픽셀 단위로 분할 하는 것
= 이미지를 픽셀 단위로 분할하여 이미지에 포함된 객체를 추출

1. 완전 합성곱 네트워크

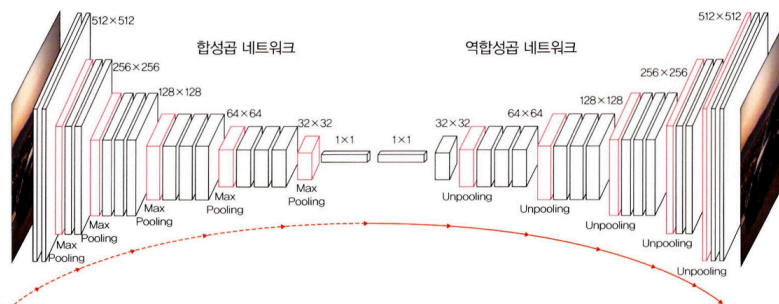
- 완전 연결층의 한계: 고정된 크기의 입력만 받아들이며, 완전 연결을 거친 후에는 위치 정보가 사라진다는 것
→ 완전 합성곱 네트워크 : 완전 연결층을 1x1 합성곱으로 대체
 - 이미지 분류에서 우수한 성능을 보인 CNN 기반 모델(AlexNet, VGG16, GoogLeNet)을 변형시켜 이미지 분할에 적합하도록 만든 네트워크
 - ex. AlexNet 아래쪽에서 사용되었던 완전연결층 세개를 1x1 합성곱으로 변환 → 남아 있는 위치정보로 히트맵 그림과 같이 고양이 위치 확인 가능



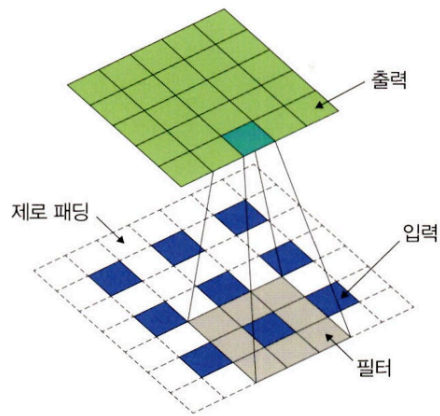
- 합성곱층으로 사용 → 입력 이미지의 크기 제약 X

2. 합성곱 & 역합성곱 네트워크

- 완전 합성곱 네트워크 단점
 - 여러 단계의 합성곱층과 풀링층을 거치면서 해상도가 낮아짐
 - 낮아진 해상도 복원을 위해 업샘플링 방식 사용 → 이미지 세부 정보 손실
- ⇒ 이를 해결하기 위해 역합성곱 네트워크 도입 = 합성곱 & 역합성곱 네트워크
- 역합성곱 : CNN의 최종 출력 결과를 원래의 입력 이미지와 같은 크기로 만들고 싶을때 사용
 - 시멘틱 분할 등에 활용, 업 샘플링(upsampling) 이라고도 함



- 특성맵 크기 증가 시키는 방식 ↔ 합성곱: 특성맵 크기 줄이는 방식
- 진행 방식
 1. 각각의 픽셀 주위에 제로 패딩 추가
 2. 패딩된 것에 합성곱 연산 수행



3. U-Net

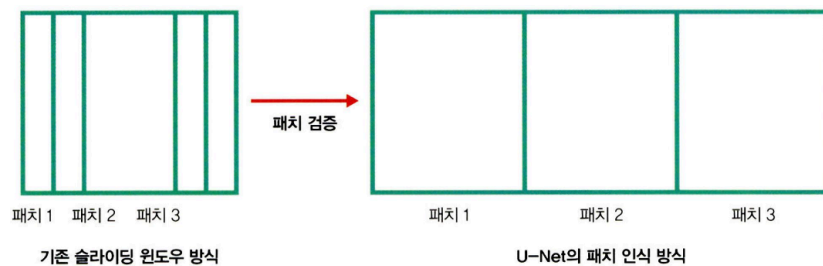
: 바이오 메디컬 이미지 분할을 위한 합성곱 신경망

- 특징

- 속도가 빠름

: 이미 검증이 끝난 패치는 건너뛴 → 속도가 빠름

↔ 기존 슬라이딩 윈도우 방식은 이전 패치(patch)에서 검증이 끝난 부분을 다음 패치에서 또 검증 → 속도가 느렸음



- 트레이드오프에 빠지지 않음

: 컨텍스트 인식과 지역화 트레이드 오프 문제를 개선

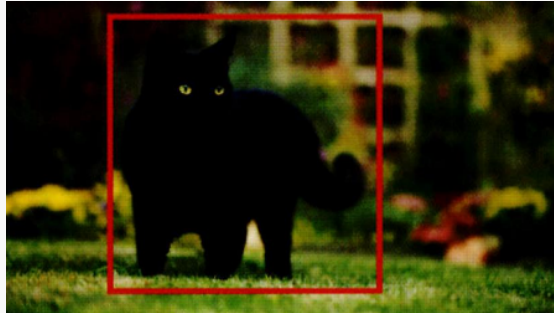
↔ 일반적으로 패치 크기 ↑ → 넓은 범위의 이미지 인식 ↑ : 컨텍스트 인식에 탁월

but 지역화에는 한계가 있음 ∵ 너무 넓은 범위를 한번에 인식하기 때문

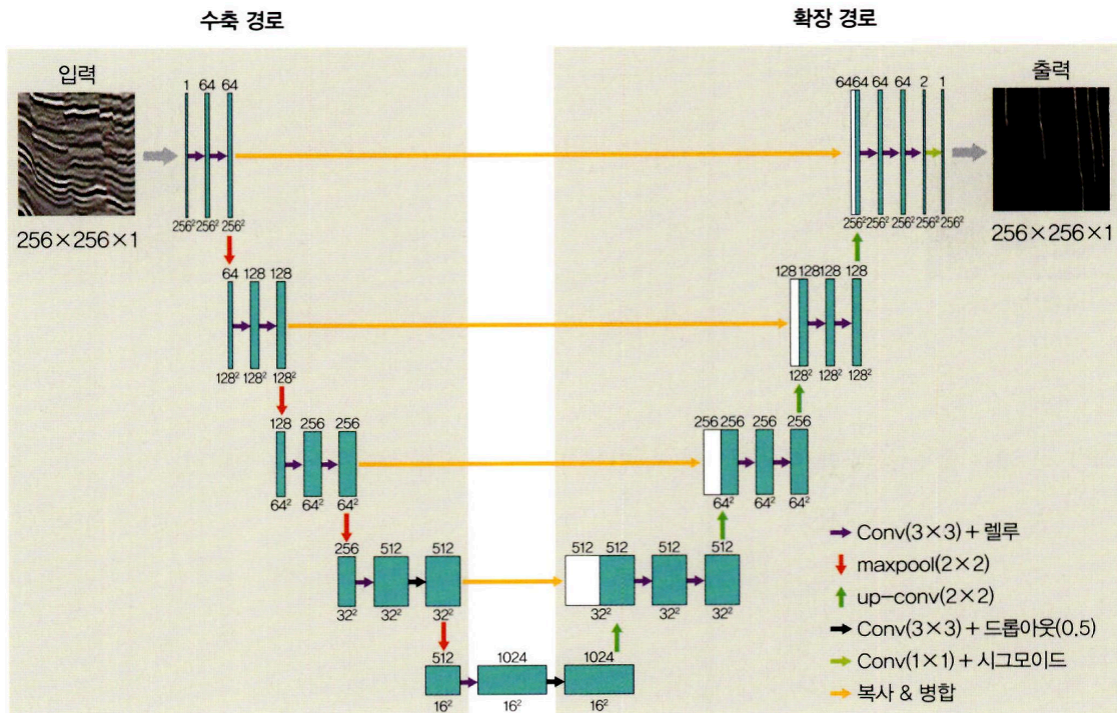


지역화

이미지 안에 객체 위치정보를 출력해주는 것으로, 주로 바운딩박스 사용
바운딩박스의 네 꼭지점 픽셀 좌표가 출력되는 것이 아닌 왼쪽 위(left top),
오른쪽 아래(right bottom)좌표 출력



- 구조
 - FCN을 기반으로 구축되었으며, 수축경로와 확장경로로 구성
 - 수축경로: 컨텍스트 포착
 - 확장경로: 특성맵 업샘플링, 수축경로에서 포착한 특성맵의 컨텍스트와 결합하여 정확한 지역화 수행
 - 3×3 합성곱이 주를 이루는데 각 합성곱 블록은 3x3 합성곱 두 개로 구성, 그 사이에 드롭아웃(dropout)이 있음
 - 그림 수축경로에서의 블록은 3x3 합성곱 두개로 구성된 것이 네개가 있는 형태
 - 각 블록은 최대풀링(maxpool) 이용, 크기를 줄이면서 다음 블록으로 넘어감
 - 그림 오른쪽 확장 경로에서 합성곱 블록에 up-conv를 앞에 붙임, 수축 과정에서 줄어든 크기를 다시 키워 가면서 합성곱 블록을 이용하는 형태
 - 크기가 다양한 이미지의 객체를 분할하기 위해 크기가 다양한 특성맵을 병합할 수 있도록 다운샘플링과 업샘플링을 순서대로 반복하는 구조

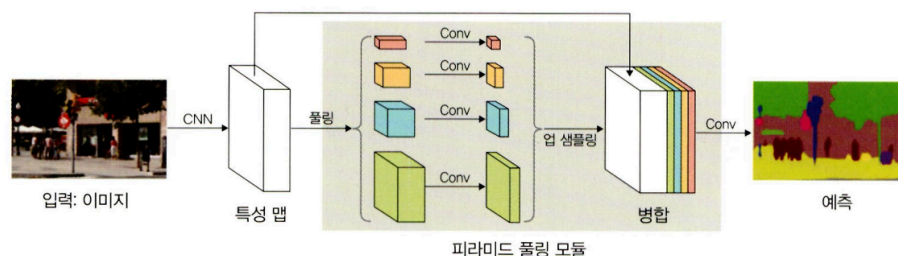


4. PSPNet

: 완전 연결층의 한계를 극복하기 위해 피라미드 풀링 모듈 추가

• 훈련과정

1. 이미지 출력이 서로 다른 크기가 되도록 여러차례 풀링 → 1x1, 2x2, 3x3, 6x6 크기로 풀링 수행 → 1x1 크기의 특성맵은 가장 광범위한 정보 담음, 각각 다른 크기의 특성맵은 서로 다른 영역들의 정보를 담음
2. 1x1합성곱을 사용하여 채널수 조정 : 풀링층 개수를 N이라고 할 때, 출력 채널수 = 입력 채널 수/N
3. 모듈의 입력 크기에 맞게 특성맵을 업샘플링 → 양선형 보간법 사용
4. 원래의 특성맵과 1~ 3과정에서 생성한 새로운 특성맵들을 병합





양선형 보간법

- 보간법

: 화소값을 할당받지 못한 영상의 품질은 안좋을 수밖에 없는데, 이때 빈화소에 값을 할당하여 좋은 품질의 영상을 만드는 방법

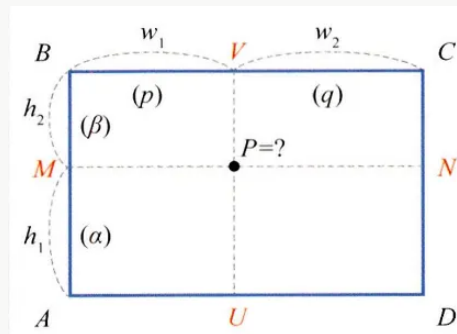
- 선형 보간법

: 원시 영상의 화소값 두 개를 사용하여 원하는 좌표에서 새로운 화소값을 계산하는 방법

- 양선형 보간법

: 화소 당 선형보간을 세 번 수행, 새롭게 생성된 화소는 가장 가까운 화소 네 개에 가중치를 곱한 값을 합해서 얻는 방법

- 예시: 직사각형의 네 꼭지점에 값이 주어져 있을 때, 사각형 내부에 있는 임의의 점(P)에 대한 값을 추정



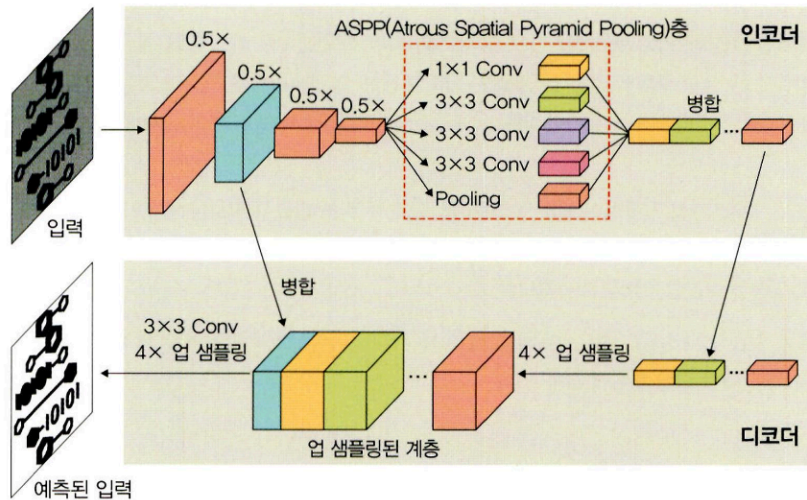
: 점 P에서 x 축 방향으로 거리를 w1, w2 / y축 방향으로 거리를 h1, h2, 이때 알려진 네 점에서 데이터 값을 A, B, C, D라고 할 때,

$$\begin{aligned} P &= q(\beta A + \alpha B) + p(\beta D + \alpha C) \\ &= q\beta A + q\alpha B + p\beta D + p\alpha C \end{aligned}$$

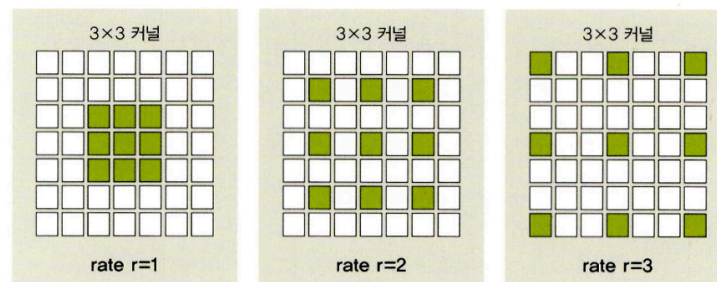
5. DeepLabv3/DeepLabv3+

: 완전 연결층의 단점을 보완하기 위해 Atrous 합성곱을 사용하는 네트워크

- 인코더-디코더 구조 + Atrous 합성곱 : 인코더에서 추출된 특성맵 해상도 제어 가능

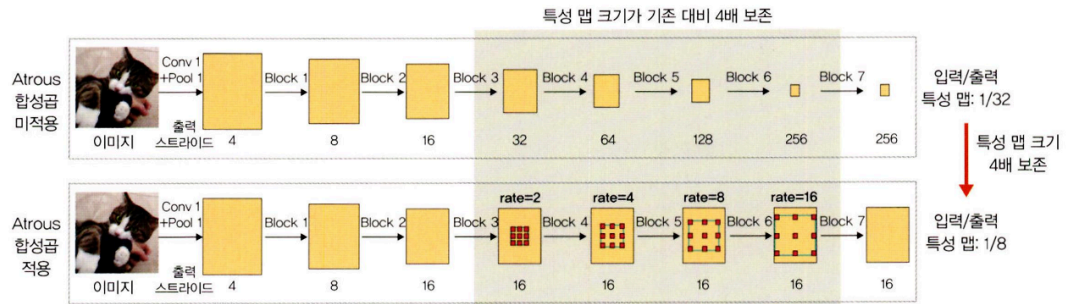


- Atrous 합성곱



- 필터 내부에 빈 공간을 둔 채로 작동
 - **rate** : 얼마나 많은 빈 공간을 가질지 결정하는 파라미터
 - rate = 1일 경우, 기존 합성곱과 동일하게 빈 공간 가짐
 - rate ↑ 빈 공간 ↑

∴ 이미지 분할에서 높은 성능을 내려면 수용 영역의 크기가 중요한데, 수용영역을 확대하여 특성을 찾는 범위를 넓게 해주기 때문
- ⇒ Atrous 합성곱 활용 시 파라미터 수를 늘리지 않으면서도 수용영역을 크게 키울 수 있기 때문에 이미지 분할 분야에서 많이 사용



- 일반적인 CNN 적용 → 출력은 입력에 비해 1/32로 줄어듦
- Atrous 합성곱 적용 → 1/8로 줄어듦
= 특성맵 크기가 기존 대비 4배 보존



수용 영역

- 외부 자극이 전체에 영향을 주는 것이 아니라 특정 영역에만 영향을 준다는 의미
- 영상에서 특정 위치에 있는 픽셀들은 그 주변에 있는 일부 픽셀과 연관성은 높지만 거리가 멀어질수록 그 영향은 감소
- 영상 전체 영역에 대해 서로 동일한 중요도를 부여하여 처리하는 대신, 특정 범위를 한정해서 처리 → 효과적으로 훈련 수행